RAHEE

Test Phase 1 Report

Project	RAHEE, an online digital version of a travel board game
Team	Team 31 (5 members)
Report period	Test Phase 1 (submitted 29 October 2026)

Summary

For Test Phase 1 we ran the automated test suite that is in the project repository on the current build. The suite covers five categories. The standard run (`npm test`) ran **60,139 checks and all of them passed**. We also ran a larger 1,000-game play-test with the computer players (171,433 moves checked) and server games for every table size from 3 to 6 players. None of these runs had a failure.

Required item	Answer (details in the section shown)
1. Test categories included	5: unit and data, regression, rule scenarios, simulated play-testing (AI players), server end-to-end (Section 1)
2. Number of tests added	60,139 checks in the standard suite (5 test scripts), plus a 1,000-game play-test and 5 extra server games (Section 2)
3. Number of tests passed	All of them: 60,139 of 60,139 in the standard suite, 1,000 of 1,000 play-test games, 5 of 5 extra server games (Section 3)
4. Number of tests failed	0 (Section 4)
5. Tools used and their outputs	Node.js test scripts, computer players (ai.js), WebSocket client (ws) with an in-memory database, GitHub Actions (Section 5)

1. Test Categories Included

#	Category	What it checks	File
1	Unit and data tests	Board data (cities, routes, the deck for each player count), the deals for 2 to 6 players, and the two scoring examples from the manual	tools/test-engine.mjs
2	Regression tests	Bugs B1 to B4 from Week 4 stay fixed: no road skips a city, roads are used before flights, only one 10 or J per hand (200 shuffles for each player count)	tools/test-engine.mjs
3	Rule scenario tests	One hand-built game situation for each rule in RULES.md: destinations, tourist visits, road trips, travelling together, Aces, budget and scoring	tools/test-rules.mjs

4	Simulated play-testing (AI players)	Complete games between computer players for 2 to 6 players and three bot levels. Every move is checked against the rules, and the run also looks for games that get stuck and for balance between the characters	tools/playtest.mjs, tools/test-engine.mjs (random games)
5	Server end-to-end (integration)	The real backend over WebSockets: hands stay private, the deck and seed are never sent, moves out of turn are refused, only the host can add bots, reconnecting with a token gives the same seat, and the scores add up	tools/test-server.mjs

All five run with one command, `npm test`, and GitHub Actions runs the same command on every push.

2. Number of Tests Added

A "check" is one test condition that can pass or fail. In the play-testing suite, one check is one move checked against the rules. The server counts change a little from run to run, because each game is random and privacy is checked on every message the server sends.

Standard suite (npm test)

Category	Test cases	Checks
1. Unit and data	Board data 15, deals 70, scoring examples 2	87
2. Regression (B1 to B4)	B3 1,000 + B2 175 + B1/B4 22	1,197
3. Rule scenarios	31 scenarios in 6 groups	31
4. Simulated play-testing	80 random games (engine script)	4,215
	300 AI games, 1,175 player seats	49,888
5. Server end-to-end	2 full games: 3 players (982) and 6 players (3,739)	4,721
Total	**5 test scripts**	**60,139**

Extended runs (same scripts, larger numbers)

Run	Test cases	Checks
AI play-testing (`node tools/playtest.mjs 1000`)	1,000 games, 4,026 player seats	171,433
Server end-to-end (`node tools/test-server.mjs N`)	5 more full games: 3 players (796), 4 players (2,045), 5 players (2,798), 6 players (3,115 and 3,577)	12,331

The server test always connects two human clients and adds at least one computer player, so its smallest game has 3 players. Running it with 2 gives a 3-player game.

3. Number of Tests Passed

Suite (command)	Passed	Failed	Result
Engine: unit, regression, 80 random games (`npm run test:engine`)	5,499	0	**Pass**
Rule scenarios (`npm run test:rules`)	31	0	**Pass**
AI play-testing, 300 games	49,888	0	**Pass**
Server end-to-end, 3 players	982	0	**Pass**
Server end-to-end, 6 players	3,739	0	**Pass**
Total (npm test)	**60,139**	**0**	**All pass**
AI play-testing, 1,000 games	171,433	0	**Pass**
Server end-to-end, 5 extra games (3 to 6 players)	12,331	0	**Pass**

4. Number of Tests Failed

0 tests failed. Every suite in `npm test` passed, all 1,000 play-test games finished with no rule broken, no stuck game and no refused move, and all extra server games passed.

5. Tools Used and Their Outputs

Tool	Used for
Node.js test scripts (no test framework)	Each script counts its checks and ends with an error code if any check fails. `npm test` (tools/run-tests.mjs) runs the engine, rules, play-testing and server suites and prints a pass or fail summary
Computer players (frontend/src/ai.js)	Planner, goal-chaser and easy bots play complete games for the play-testing and server tests. They see only what a real player's browser is sent
ws (WebSocket client) and an in-memory database (DB_DRIVER=memory)	The server test connects real WebSocket clients to the real backend, so no MySQL server is needed
GitHub Actions (.github/workflows/test.yml)	Runs `npm test` on every push and pull request and keeps the play-test report as a download

Output: npm test

```
=== engine ===
80 random games, average 6.1 rounds, best score 60
Checks by section:
  Board data sanity.....................................     15
  Deals for 2 to 6 players..............................     70
  B3: one high card per hand, 200 shuffles x 5 player counts....  1,000
  B2: roads before flights..............................    175
  B1/B4: no road jumps over a city......................     22
  Scoring examples from the manual......................      2
  Random complete games.................................  4,215
  Total.................................................  5,499
all 5,499 checks passed

=== rules ===
Destinations
  ok   ending a move on a destination does not complete it
  ok   paying the ₹2,000 visit completes the destination
  ok   the fifth completed destination triggers the end at once
  ok   ending where another traveller already is gives no memory
Tourist visits
  ok   the client cannot claim the shared price on an ordinary visit
  ok   no visit while still moving, and not for someone else's city
  ok   a place scores once per player
Road trips
  ok   the driver's own face-up card is never offered
  ok   the driver cannot flip their own card
  ok   the card must belong to someone on the trip
  ok   road trip off when the card's owner declines
  ok   manual worked example: 4 steps, costs shared, memories
  ok   a player already at the trip's end gets no road-trip memory
  ok   shared tourism ₹1,000 for those still aboard; none for a rider dropped off
  ok   a rider who got off partway forfeits the shared price
  ok   undo on a road trip refunds everyone exactly and brings riders back
  ok   a confirmed road trip that hasn't moved can be called off; the card goes face-up again
  ok   log lines name the player they are about
  ok   a road-trip companion can take or turn down the shared stamp themselves
Travelling together
  ok   offered whenever someone is in your city — no face-up card needed
  ok   uses the mover's own card: nothing flipped, no road-trip memory, rider gets nothing
  ok   an Ace works as usual (memory mode gives the mover one more)
  ok   shared ₹1,000 stamp; then the turn ends — no road trip after
  ok   nobody comes: back to the start of the turn, card still in hand
  ok   called off before moving returns to the start too
Aces
  ok   two Aces: swap one for a card from the undealt pile
  ok   one Ace cannot be swapped
  ok   no cards left: reshuffle deals every hand again
  ok   Ace as booster: efficient travel needs the +1
Budget and scoring
  ok   starting budget is ₹20,000 (RULES.md)
  ok   manual turn example: Jaipur, 5♣ to Gwalior costs ₹1,000 + ₹2,000 stay
31 passed, 0 failed

=== playtest ===
300/300 games completed in 13.5s — avg 5.6 rounds, avg score 61.7
49,888 moves checked, 1,175 player seats, 0 moves refused
no rule violations, stalls or refused moves
report: playtest-report/test-run/report.md

=== server (3 players) ===
  game over after 6 rounds: Asha 145, Bilal 110, Meera (AI) 95
982 server checks, all passed

=== server (6 players) ===
```

```
  game over after 5 rounds: Bilal 140, Asha 135, Meera (AI) 115, Rohan (AI) 105, Arjun (AI) 95, Kavya
(AI) 20
3739 server checks, all passed

=== summary ===
  PASS  engine  (0.6s)
  PASS  rules  (0.3s)
  PASS  playtest  (14.1s)
  PASS  server (3 players)  (6.9s)
  PASS  server (6 players)  (6.7s)
```

Output: 1,000-game play-test and extra server games

```
$ node tools/playtest.mjs 1000
1000/1000 games completed in 47.1s — avg 5.6 rounds, avg score 63.1
171,433 moves checked, 4,026 player seats, 0 moves refused
no rule violations, stalls or refused moves
report: playtest-report/report.md

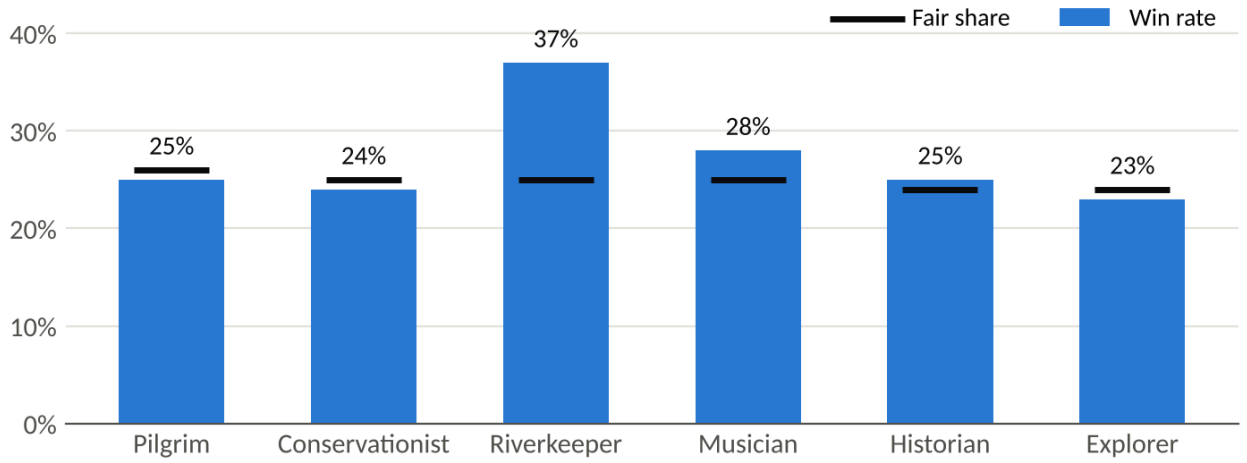
$ node tools/test-server.mjs 2
  game over after 5 rounds: Asha 120, Meera (AI) 115, Bilal 100
796 server checks, all passed
$ node tools/test-server.mjs 4
  game over after 6 rounds: Bilal 160, Meera (AI) 150, Asha 130, Arjun (AI) 115
2045 server checks, all passed
$ node tools/test-server.mjs 5
  game over after 6 rounds: Meera (AI) 140, Asha 135, Bilal 125, Arjun (AI) 125, Kavya (AI) 55
2798 server checks, all passed
$ node tools/test-server.mjs 6
  game over after 5 rounds: Asha 105, Bilal 105, Arjun (AI) 105, Meera (AI) 100, Rohan (AI) 80, Kavya
(AI) 20
3115 server checks, all passed
$ node tools/test-server.mjs 6
  game over after 5 rounds: Meera (AI) 160, Arjun (AI) 145, Bilal 140, Asha 100, Rohan (AI) 75, Kavya
(AI) 10
3577 server checks, all passed
```

Output: 1,000-game play-test report (playtest-report/report.md)

1,000 of 1,000 games completed, 2 to 6 players, three bot levels, budget ₹20,000, seed 1. No rule violations, stuck games or refused moves.

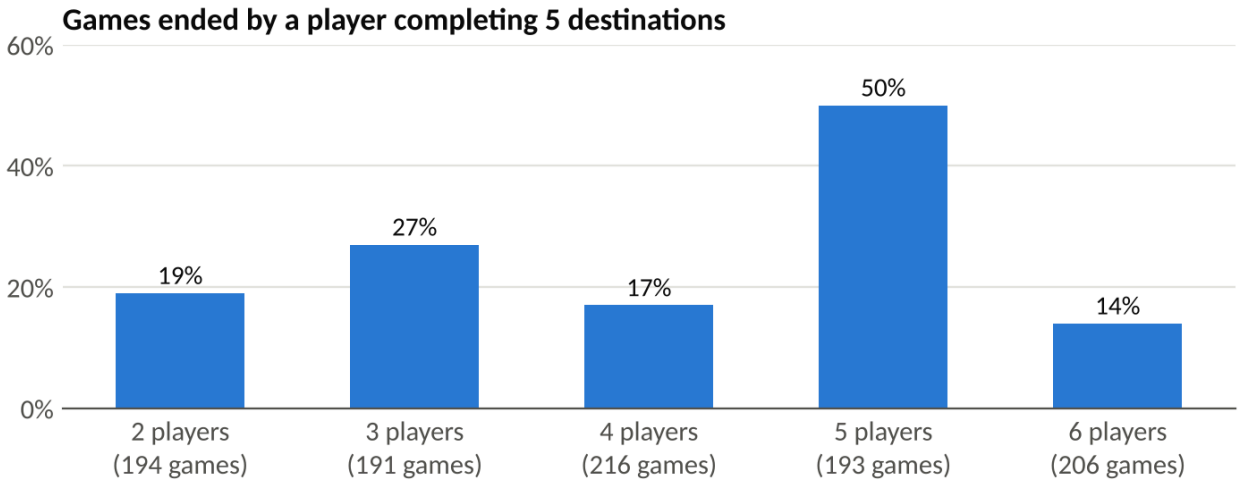
A typical game	Per game	A typical player	Per player
Rounds	5.6	Final score	63.1
Actions sent	171.4	Stamps	4.7
Flights taken	3.5	Destinations completed	2.4
Ship crossings	0.2	Completed all 5	10%
Road trips offered / taken	1.8 / 4.1	Memories	4.5
Invitations declined	12%	Money spent	₹14,769
Games ended by 5 destinations	25%	Went over budget	12%
Ace swaps / reshuffles (all games)	504 / 76		

Win rate by character, 1,000 AI games



Character	Seats	Win rate	Fair share	Avg score	Destinations	Stamps	Over budget
Pilgrim	695	25%	26%	63.7	2.3	4.5	9%
Conservationist	757	24%	25%	64.9	2.3	4.6	10%
Riverkeeper	656	37%	25%	67.7	2.8	5.0	10%
Musician	716	28%	25%	66.1	2.7	4.6	12%
Historian	594	25%	24%	52.7	2.2	4.9	19%
Explorer	608	23%	24%	62.0	1.9	4.6	10%

Fair share is 1 divided by the number of players at the table. Five of the six characters win close to their fair share. The Riverkeeper wins 37% of its games against a fair share of 25%. The Historian has the lowest average score and goes over budget most often (19%). The computer players are a play-testing aid and do not play like people, so these numbers show where to look rather than a final answer.



Players	Games	Rounds	Winner's score	Avg score	Ended by destinations
2	194	6.1	93.6	57.9	19%
3	191	5.9	107.3	60.0	27%
4	216	5.4	111.6	63.6	17%
5	193	5.7	129.0	69.5	50%
6	206	4.9	115.2	61.0	14%

By bot level

Level	How it plays	Seats	Win rate	Avg score	Destinations
greedy (planner)	Plans each turn for the most points	1,344	52%	103.9	3.0
goals (goal-chaser)	Chases its character's destinations	1,337	29%	90.7	4.0
easy	Random legal moves	1,345	0%	-5.0	0.2

Examples from the test code

Three of the checks, copied from the test scripts, to show what a single check looks like.

Regression B1 and B2 (tools/test-engine.mjs)

```
check(!has("bhopal", "lucknow"), "B1: there is no printed Bhopal-Lucknow road (it is Bhopal-Gwalior +
Gwalior-Lucknow)");
check(stops.includes("gwalior"), `B1: Sawai Madhopur -> Madikeri (4 steps) should pass through
Gwalior, got ${stops.join(" > ")}`);
check(route && route.edges.every((e) => e.route.type !== "flight"), "B2: New Delhi -> Mumbai should be
by road, not a flight");
```

Rule scenario: shared ₹1,000 stamp on a road trip (tools/test-rules.mjs)

```
test("shared tourism ₹1,000 for those still aboard; none for a rider dropped off", () => {
  const s = tripSetup();
  E.offerRoadTrip(s);
  E.proposeRoadTrip(s, graph, { companions: [1], owner: 1, cardId: "4♦", destination: "gwalior" });
  E.respondRoadTrip(s, graph, 1, { type: "accept", stop: "gwalior" });
  E.confirmRoadTrip(s);
  E.stepTo(s, graph, "gwalior");
  E.endRoadTrip(s);
  const opts = E.visitOptions(s);
  assert(opts.length === 2 && opts.every((o) => o.price === econ.sharedTouristVisit), "shared price
not offered to both");
});
```

Server: privacy on every message, and moves out of turn refused (tools/test-server.mjs)

```
if (m.type === "state") {
  check(!("board" in m.state) && !("leftover" in m.state) && !("seed" in m.state), `${name}: server
sent the board, deck or seed`);
  m.state.players.forEach((p, i) => {
    if (i !== c.seat) check(p.hand === undefined && typeof p.handCount === "number", `${name}: saw
player ${i}'s hand`);
  });
}
...
outsider.send({ type: "action", name: "playCard", args: { cardId:
outsider.state.players[outsider.seat].hand[0].id } });
await until(() => outsider.errors.length > before);
check(outsider.errors.slice(before).some((e) => /isn't your turn/.test(e)), "an out-of-turn move was
not refused");
```

6. How to Re-run the Tests

```
cd backend && npm install && cd ..     # once
npm test                               # engine, rules, 300-game play-test, server (3 and 6 players)
node tools/playtest.mjs 1000           # 1,000-game play-test, report in playtest-report/report.md
node tools/test-server.mjs 5           # one server game for 5 players (3 to 6)
```

7. Date of the Current Build

Build tested	29 October 2026, main branch